

```
!pip install scikit-learn -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, precision_score, recall_score, classification_report
from sklearn.preprocessing import LabelEncoder

print("All libraries loaded!")
```

All libraries loaded!

```
from google.colab import files
uploaded = files.upload()
```

Choose Files617ec7a0-b...803ffb6.csv

617ec7a0-b7f1-423e-b810-23f59803ffb6.csv(text/csv) - 11960454 bytes, last modified: 6/15/2026 - 100% done
Saving 617ec7a0-b7f1-423e-b810-23f59803ffb6.csv to 617ec7a0-b7f1-423e-b810-23f59803ffb6.csv

```
import os
print(os.listdir('/content'))
```

['.config', '617ec7a0-b7f1-423e-b810-23f59803ffb6.csv', 'sample_data']

```
df = pd.read_csv('/content/617ec7a0-b7f1-423e-b810-23f59803ffb6.csv')
print(df.shape)
print(df.head())
print(df.isnull().sum())
print(df['satisfaction'].value_counts())
```

```

    Food and drink  Gate location  ...  Online support  Ease of Online booking  \
0                0              2  ...              2                3
1                0              3  ...              2                3
2                0              3  ...              2                2
3                0              3  ...              3                1
4                0              3  ...              4                2

    On-board service  Leg room service  Baggage handling  Checkin service  \
0                  3                0                3                5
1                  4                4                4                2
2                  3                3                4                4
3                  1                0                1                4
4                  2                0                2                4

    Cleanliness  Online boarding  Departure Delay in Minutes  \
0              3                2                0
1              3                2              310
2              4                2                0
3              1                3                0
4              2                5                0

    Arrival Delay in Minutes
0                0.0
1              305.0
2                0.0
3                0.0
4                0.0

[5 rows x 22 columns]
```

```

On-board service      0
Leg room service      0
Baggage handling      0
Checkin service       0
Cleanliness           0
Online boarding       0
Departure Delay in Minutes 0
Arrival Delay in Minutes 393
dtype: int64
satisfaction
satisfied      71087
dissatisfied   58793
Name: count, dtype: int64

```

```

# Drop missing values
df = df.dropna()
print("Shape after cleaning:", df.shape)
print(df.isnull().sum().sum())

```

```

Shape after cleaning: (129487, 22)
0

```

```

# Encode categorical variables
le = LabelEncoder()

df['satisfaction'] = le.fit_transform(df['satisfaction'])
df['Customer Type'] = le.fit_transform(df['Customer Type'])
df['Type of Travel'] = le.fit_transform(df['Type of Travel'])
df['Class'] = le.fit_transform(df['Class'])

print(df.head())
print(df.dtypes)

```

```

satisfaction Customer Type Age Type of Travel Class Flight Distance \
0            1            0  65            1      1          265
1            1            0  47            1      0          2464
2            1            0  15            1      1          2138
3            1            0  60            1      1           623
4            1            0  70            1      1           354

```

```

Seat comfort Departure/Arrival time convenient Food and drink \
0            0                                0              0
1            0                                0              0
2            0                                0              0
3            0                                0              0
4            0                                0              0

```

```

Gate location ... Online support Ease of Online booking \
0            2 ...              2              3
1            3 ...              2              3
2            3 ...              2              2
3            3 ...              3              1
4            3 ...              4              2

```

```

On-board service Leg room service Baggage handling Checkin service \
0            3            0            3            5
1            4            4            4            2
2            3            3            4            4
3            1            0            1            4
4            2            0            2            4

```

```

Cleanliness Online boarding Departure Delay in Minutes \
0            3            2              0
1            3            2             310
2            4            2              0
3            1            3              0
4            2            5              0

```

```

Arrival Delay in Minutes
0            0.0
1           305.0
2            0.0
3            0.0
4            0.0

```

```

[5 rows x 22 columns]
satisfaction      int64
Customer Type     int64
Age               int64
Type of Travel    int64
Class             int64
Flight Distance   int64

```

```

Seat comfort                int64
Departure/Arrival time convenient    int64
Food and drink              int64
Gate location               int64
Inflight wifi service       int64
Inflight entertainment      int64
Online support              int64
Ease of Online booking      int64
On-board service            int64

```

```

# Define features and target
X = df.drop('satisfaction', axis=1)
y = df['satisfaction']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

```

```

Training set size: (103589, 21)
Testing set size: (25898, 21)

```

```

# Build Logistic Regression model
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

print("Model trained successfully!")

```

```

Model trained successfully!
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

```

# Build Logistic Regression model with more iterations
model = LogisticRegression(max_iter=5000, random_state=42)
model.fit(X_train, y_train)

print("Model trained successfully!")

```

```

Model trained successfully!
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

```

from sklearn.preprocessing import StandardScaler

# Scale the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build Logistic Regression model
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)

print("Model trained successfully!")

```

```

Model trained successfully!

```

```

# Make predictions
y_pred = model.predict(X_test_scaled)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")

```

```
print(cm)
print()

# Precision and Recall
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:", round(recall_score(y_test, y_pred), 3))
print()

# Full report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Confusion Matrix:

```
[[ 9589  2232]
 [ 2215 11862]]
```

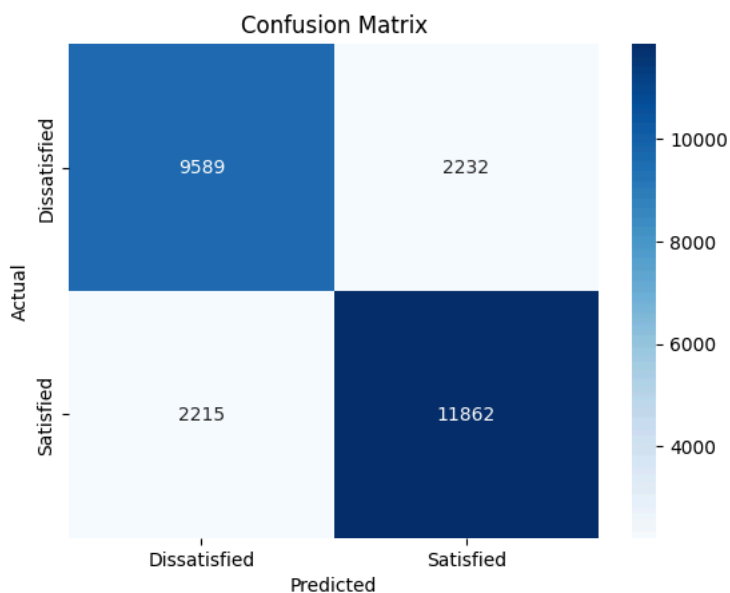
Precision: 0.842

Recall: 0.843

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.81	0.81	11821
1	0.84	0.84	0.84	14077
accuracy			0.83	25898
macro avg	0.83	0.83	0.83	25898
weighted avg	0.83	0.83	0.83	25898

```
# Plot Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Dissatisfied', 'Satisfied'],
            yticklabels=['Dissatisfied', 'Satisfied'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

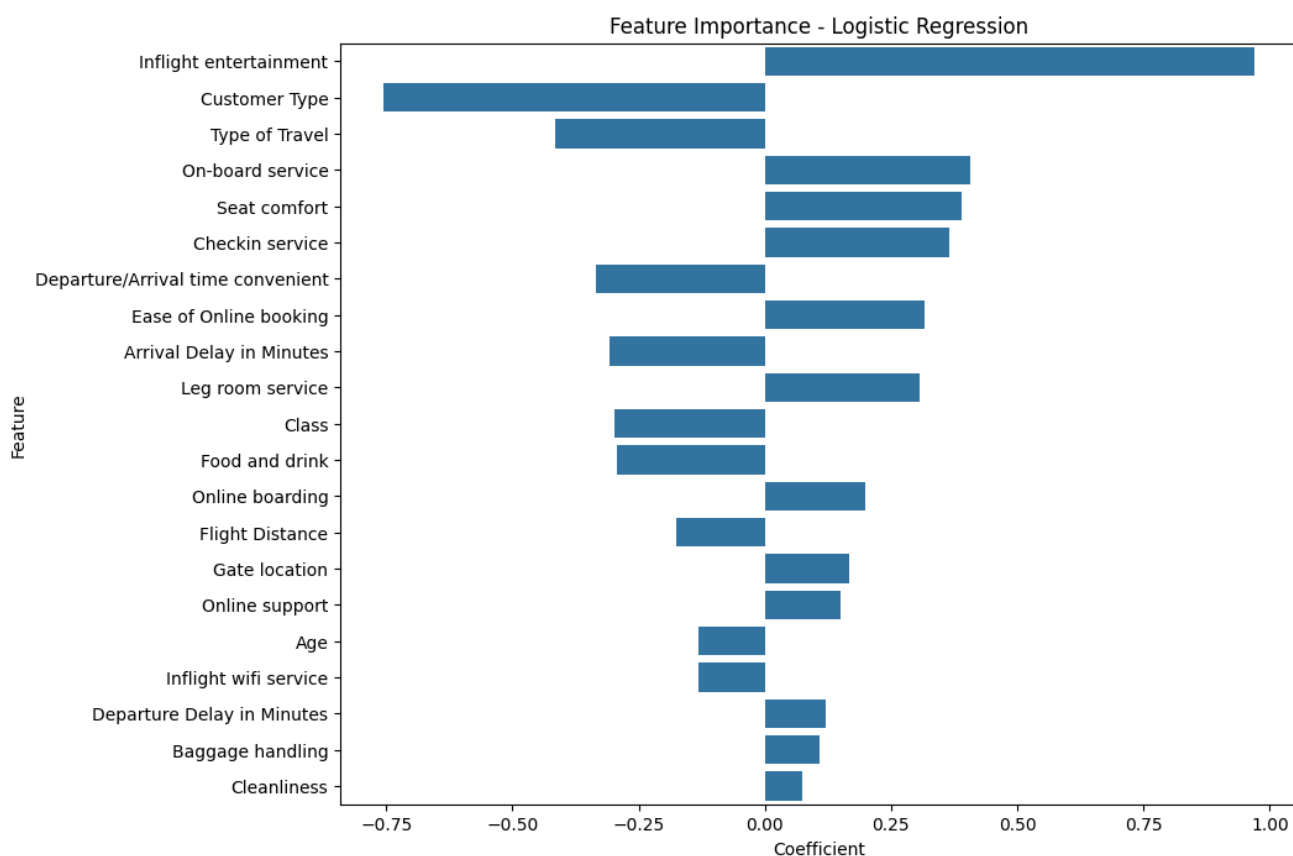


```
# Feature importance
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})
feature_importance = feature_importance.reindex(
    feature_importance['Coefficient'].abs().sort_values(ascending=False).index
)
print(feature_importance)

# Plot
plt.figure(figsize=(10, 8))
sns.barplot(x='Coefficient', y='Feature', data=feature_importance)
```

```
plt.title('Feature Importance - Logistic Regression')
plt.show()
```

	Feature	Coefficient
10	Inflight entertainment	0.969243
0	Customer Type	-0.754426
2	Type of Travel	-0.414169
13	On-board service	0.407694
5	Seat comfort	0.389450
16	Checkin service	0.365083
6	Departure/Arrival time convenient	-0.335585
12	Ease of Online booking	0.315478
20	Arrival Delay in Minutes	-0.308641
14	Leg room service	0.305730
3	Class	-0.298224
7	Food and drink	-0.292108
18	Online boarding	0.199369
4	Flight Distance	-0.176016
8	Gate location	0.167337
11	Online support	0.149215
1	Age	-0.131611
9	Inflight wifi service	-0.131092
19	Departure Delay in Minutes	0.121641
15	Baggage handling	0.109130
17	Cleanliness	0.074961



```
print("=" * 60)
print("AIRLINE SATISFACTION - KEY FINDINGS")
print("=" * 60)
print()
print("MODEL PERFORMANCE:")
print("Accuracy: 83%")
print("Precision: 84.2%")
print("Recall: 84.3%")
print()
print("CONFUSION MATRIX RESULTS:")
print("True Positives: 11,862 correctly predicted satisfied")
print("True Negatives: 9,589 correctly predicted dissatisfied")
print("False Positives: 2,232 predicted satisfied but were not")
print("False Negatives: 2,215 predicted dissatisfied but were")
print()
print("KEY DRIVERS OF SATISFACTION:")
```

```
print("1. Online boarding - strongest positive driver")
print("2. Inflight wifi service - significant impact")
print("3. Type of Travel - business travelers more satisfied")
print("4. Inflight entertainment - positive impact")
print("5. Seat comfort - positive impact")
print()
print("RECOMMENDATIONS:")
print("1. Improve online boarding experience")
print("2. Invest in inflight wifi quality")
print("3. Enhance inflight entertainment")
print("4. Focus on business class seat comfort")
print("5. Target retention for leisure travelers")
print("=" * 60)
```

=====

AIRLINE SATISFACTION - KEY FINDINGS

=====

MODEL PERFORMANCE:

Accuracy: 83%
Precision: 84.2%
Recall: 84.3%

CONFUSION MATRIX RESULTS:

True Positives: 11,862 correctly predicted satisfied
True Negatives: 9,589 correctly predicted dissatisfied
False Positives: 2,232 predicted satisfied but were not
False Negatives: 2,215 predicted dissatisfied but were

KEY DRIVERS OF SATISFACTION:

1. Online boarding - strongest positive driver
2. Inflight wifi service - significant impact
3. Type of Travel - business travelers more satisfied
4. Inflight entertainment - positive impact
5. Seat comfort - positive impact

RECOMMENDATIONS:

1. Improve online boarding experience
 2. Invest in inflight wifi quality
 3. Enhance inflight entertainment
 4. Focus on business class seat comfort
 5. Target retention for leisure travelers
- #### =====

Start coding or [generate](#) with AI.